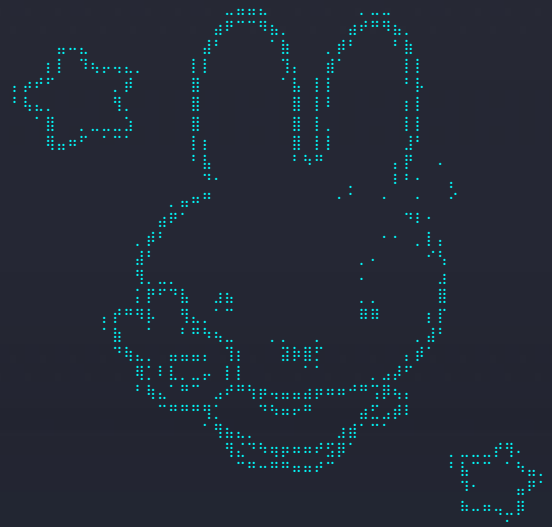
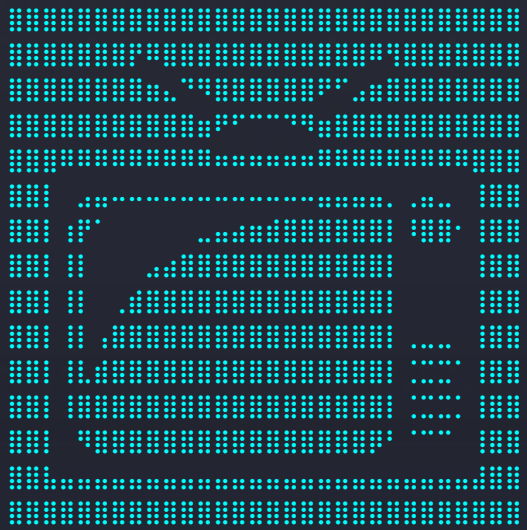
# SELECTOR DE SERIES

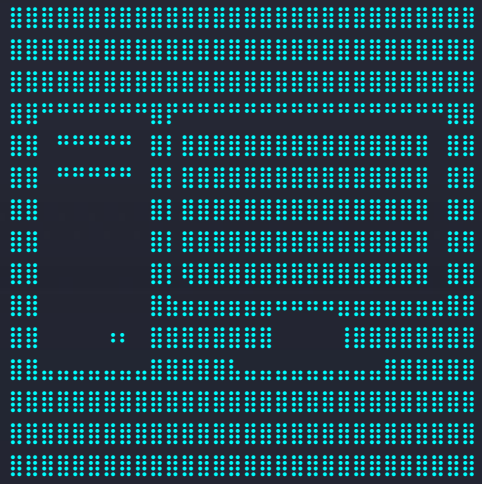




- Integrantes del grupo: (Litia Calderón Maceira, María Montobbio Ortiz)

# ¿De qué trata?:

- ¿Qué pasa cuando no tienes tiempo para ver todas las series que querrías?



「(°ヘ°)

- Cuando **no** tienes **tiempo suficiente**, hay que **priorizar qué series ver**, por lo tanto, hemos creado un programa **capaz de ordenar series por su duración o valoración**.

Ordenar por duración

```
Breaking Bad 2293 Min 5/5
Jojo's Bizzarre Adventure: Stone Ocean 0937 Min 3/5
Milya Broken 0253 Min 1/5
I cannot Drown 0665 Min 4/5
.flow 0663 Min 0/5
```

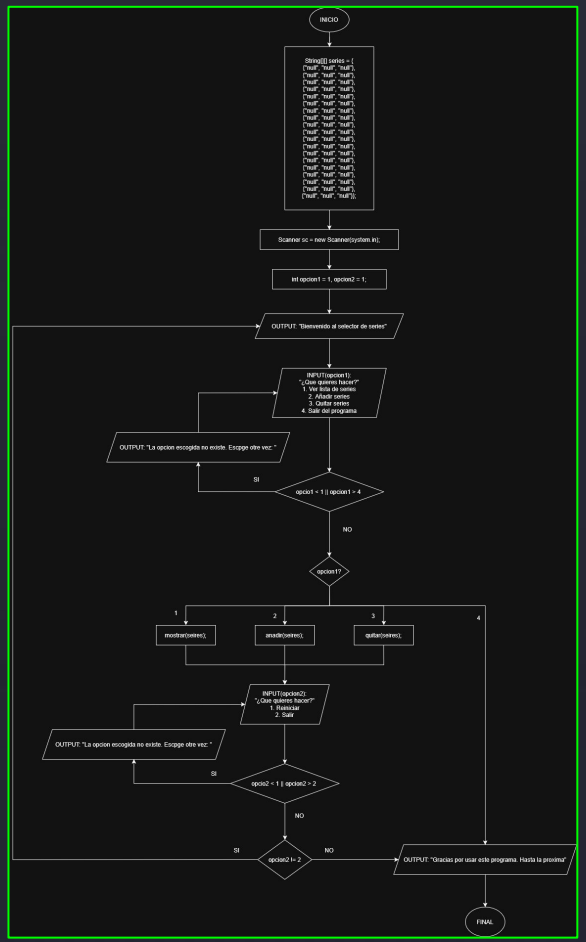
↓

```
Milya Broken 0253 Min 1/5
.flow 0663 Min 0/5
I cannot Drown 0665 Min 4/5
Jojo's Bizzarre Adventure: Stone Ocean 0937 Min 3/5
Breaking Bad 2293 Min 5/5
```

(˚O˚)!!

# Flowchart:

Flowchart main

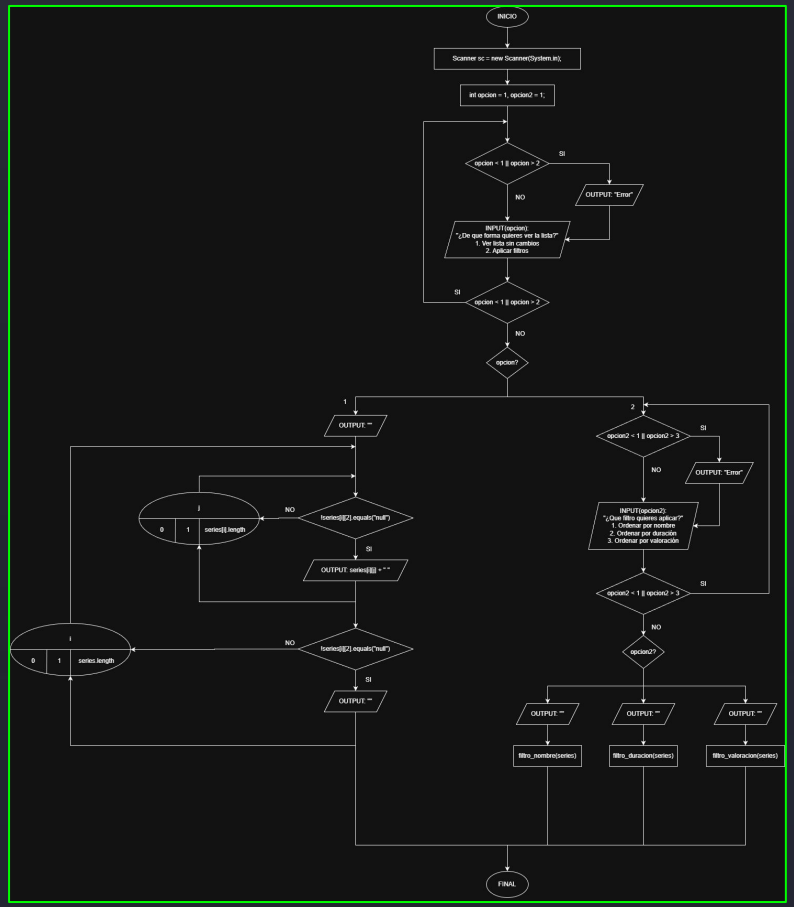



- És el menú principal, el que más verá el usuario y desde donde se llaman a las distintas funciones

( ╭ʊ_•́)

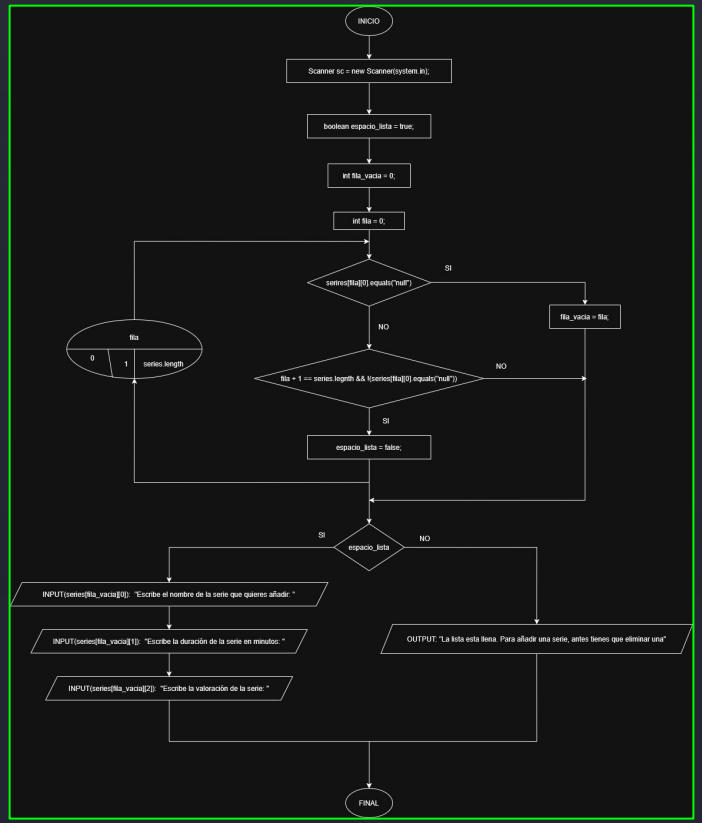
# Flowchart Mostrar

- Se pregunta y se muestra la lista de series ordenado de la forma que quiere el usuario.




(/ /ᘓ ᴗ ᘓ/ / /)

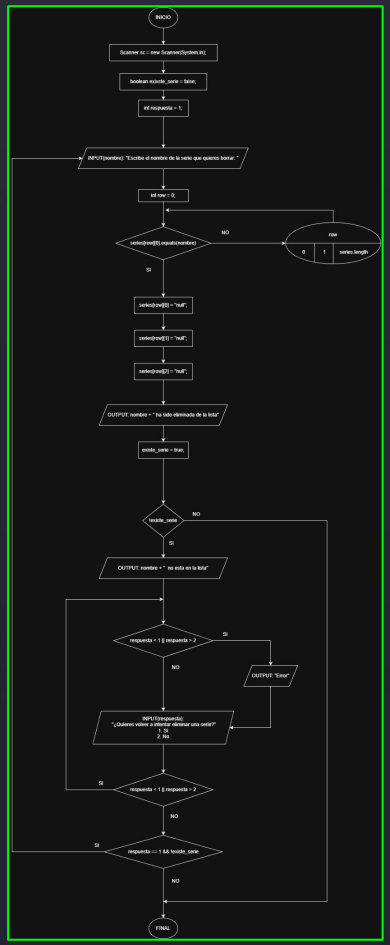
# Flowchart Añadir:




- Así planeamos la función de añadir a nuestro proyecto
- Sustituye una fila con “null” por información de la nueva serie

(ɔ° - °ς)

# Flowchart Quitar




- Así planeamos la función de quitar una serie de la lista
- Convierte todos los elementos en “null” y luego se ignoran a la hora de mostrarlos en pantalla

# Almacenamiento:

- ¿Y dónde se almacenan los datos?

```
String[][] series = { { "Breaking Bad", "2930 Min", "5/5" },
        { "Jojo's Bizzarre Adventure: Stone Ocean", "937 Min", "3/5" },
        { "Milya Broken", "253 Min", "1/5" },
        { "I cannot Drown", "665 Min", "4/5" },
        { ".flow", "663 Min", "0/5" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" },
        { "null", "null", "null" }, };
```

# Colecciones importadas:

- java.util.Arrays;

```
import java.util.Arrays;
```

- Esta librería se ha usado específicamente para poder ordenar una array2D por filas

```
    case 2:
    // Esto ayuda a ordenar filas o columnas en una array2D comparando dos elementos...
Arrays.sort(series, (a, b) -> a[1].compareTo(b[1]));

    // Bucle para mostrar las filas ordenadas por duración... aunque sea una String
    for (int i = 0; i < series.length; i++){
        for (int j = 0; j < series[i].length; j++){
            if (!series[i][2].equals(anObject: "null")){
                System.out.print(series[i][j] + " ");
            }
            }
            if (!series[i][2].equals(anObject: "null")){
                System.out.println();
            }
}
```

- java.util.Scanner;

```
Scanner sc = new Scanner(System.in);
```

- Era necesario poder escanear el **input del usuario**

- Ejemplos:

Modificar Tabla

```
System.out.println(x: "Escribe el nombre de la series que quieres añadir: ");
series[fila_vacia][0] = sc.nextLine();
System.out.println(x: "Escribe la duración de la series en minutos: ");
series[fila_vacia][1] = sc.nextLine();
System.out.println(x: "Escribe la valoración de la serie: ");
series[fila_vacia][2] = sc.nextLine();
}
```

Opciones

```
opcion1 = sc.nextInt();

if(opcion1 == 4) break;

switch (opcion1) {
```

# Implementación técnica: Switch Case

- Para la organización de los menús, principalmente se ha utilizado un **switch case:**

MENÚ PRINCIPAL

```java
switch (opcion1) {
    case 1:
        mostrar(series);
        break;
    case 2:
        anadir(series);
        break;
    case 3:
        quitar(series);
        break;
    default:
        break;
}
```

SUB MENÚ

```java
int opcion = sc.nextInt();
switch (opcion) {
            // Mostrar lista sin alterar
    case 1:
            for (int i = 0; i < series.le
                for (int j = 0; j < serie
                    if (!series[i][2].equ
                    System.out.print(seri
                    }
    }
                    if (!series[i][2].equ
                    System.out.println();
                    }
}
    break;
            // FILTROS
    case 2:
```

# Bucles (for, while)

- Los bucles for han sido muy útiles para mostrar la tabla en pantalla

```
    for (int i = 0; i < series.length; i++){
        for (int j = 0; j < series[i].length; j++){
            if (!series[i][2].equals(anObject: "null")){
                System.out.print(series[i][j] + " ");
            }
            }
            if (!series[i][2].equals(anObject: "null")){
                System.out.println();
            }
}
```

- Y el while ha servido para mantener poder cerrar el programa después de cada acción

```
    System.out.println(x: "¿Que quieres hacer?");
    System.out.println(x: "1. Reiniciar");
    System.out.println(x: "2. Salir");

    opcion2 = sc.nextInt();

} while(opcion2 != 2);
```

# Iterativos (if)

- Los iterativos, a parte de para la gestión de errores, han sido útiles para éstas funciones donde ignora las filas que no tienen serie asignada

```
case 1:
    System.out.println();
    for (int i = 0; i < series.length; i++) {
        for (int j = 0; j < series[i].length; j++) {
            if (!series[i][2].equals(anObject: "null")) {
                System.out.print(series[i][j] + " ");
            }
        }
        if (!series[i][2].equals(anObject: "null")) {
            System.out.println();
        }
    }
    break;
```

## Métodos

- Para mantener un código ordenado, hemos utilizado diferentes métodos para las opciones que te proporciona el programa

- Por cada caso de la variable opción, se llama a una función

```
switch (opcion1) {
    case 1:
        mostrar(series);
        break;
    case 2:
        anadir(series);
        break;
    case 3:
        quitar(series);
        break;
    default:
        break;
}
```

# Gestión de errores (if)

- Para la gestión de errores se han usado una serie de iterativos para evitar inputs no deseados

```
if (opcion1 < 1 || opcion1 > 4) System.out.println(x: "\nLa opcion escigida no existe. Escoge otra vez:");
```

```
if (!existe_serie) {
    System.out.println(nombre + " no esta en la lista\n");
```

```
if (opcion < 1 || opcion > 2)
    System.out.println(x: "\nError\n");
```

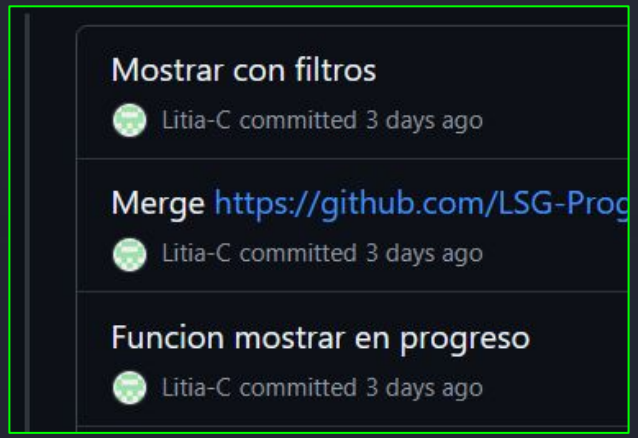
<(˘ ˘ ˘)>

# Test del programa

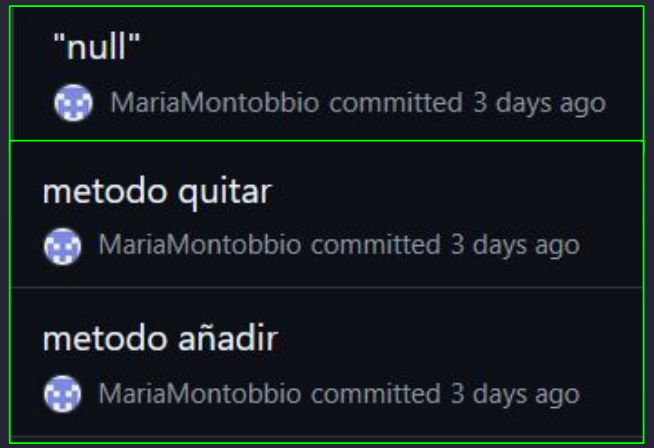
# Gestión de versiones:

- Durante el proyecto, hemos trabajado en dos ramas distintas para luego unirlas en la rama principal

Rama litia




Rama maria




* Solo se muestran unos pocos commits significativos

- Una vez se acabaron las funciones principales y se unieron para que funcionen bien en conjunto, María se dedicó a la gestión de errores

## Rama main

Release
Litia-C committed 2 minutes ago

Merge completo Ver Lista, Añadir y Salir
Litia-C committed 3 days ago

control errores quitar
MariaMontobbio committed 2 days ago

control errores escoger opcion manu principal
MariaMontobbio committed 2 days ago

añadir: Error si la lista esta llena
MariaMontobbio committed 2 days ago

* Solo se muestran unos pocos commits significativos

# Reflexión final: Desafíos superados y mejoras

- Fue un desafío **crear un programa** que cumpliera su propósito **sin poder usar la librería de ArrayList**

- Posiblemente poder implementar la librería de ArrayList con una Array2D usando clases mejoraría mucho el programa al poder **usar una tabla dinámica**

( „° ▭ °) !!

## Valoración final

- Ha habido buena coordinación y trabajo en equipo, con buen reparto de tareas y ayuda mutua
- Se ha consolidado como usar git, se han implementado métodos por primera vez, se han aprendido un uso de la librería de array

`▨ω▨´-✧

# ¡Gracias por vuestra atención!

(*^‿^)ノ*.°